

同濟大學

TONGJI UNIVERSITY

机器学习实验报告

实验名称	MNIST 手写数字识别算法对比研究
学 院	计算机科学与技术学院
姓 名	李盛鹏, 殷和尘, 程亦诚
日 期	2025 年 11 月 29 日

1 实验介绍

1.1 实验背景与意义

手写数字识别是模式识别和机器学习领域的经典问题，在邮政自动分拣、银行支票识别、文档数字化等场景中具有重要应用价值。MNIST 数据集作为该领域的标准基准，包含 60,000 个训练样本和 10,000 个测试样本，因其规模适中、图像统一（28×28 像素）、类别均衡的特点，成为比较不同算法性能的理想平台。

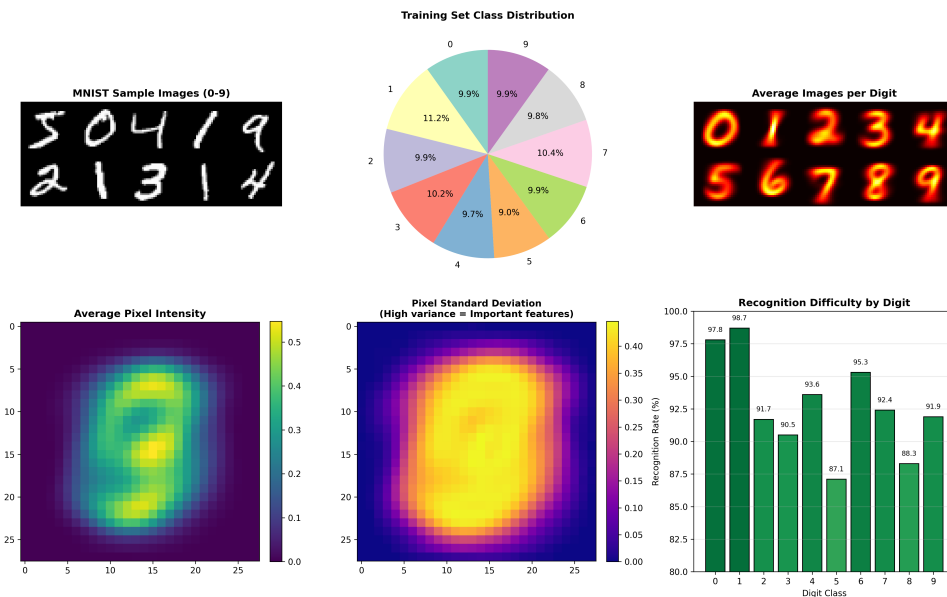


图 1.1 MNIST 数据集分析：样本分布与可视化

本文通过系统性比较深度学习与多种传统机器学习方法在 MNIST 任务上的表现，旨在为实际应用中的算法选择提供科学依据。

1.2 实验内容与目标

本实验选取五种代表性算法进行系统比较：卷积神经网络（CNN）、全连接神经网络、支持向量机、随机森林和逻辑回归。通过统一实验环境评估各算法在准确率、训练时间、模型复杂度等方面的表现，并深入分析其适用场景。

主要目标包括：（1）评估多种传统方法的效果与特点；（2）验证深度学习在图像识别中的性能优势；（3）为不同应用场景提供算法选择指导；（4）揭示各类算法的内在特点和局限性。

2 实验设计与实现

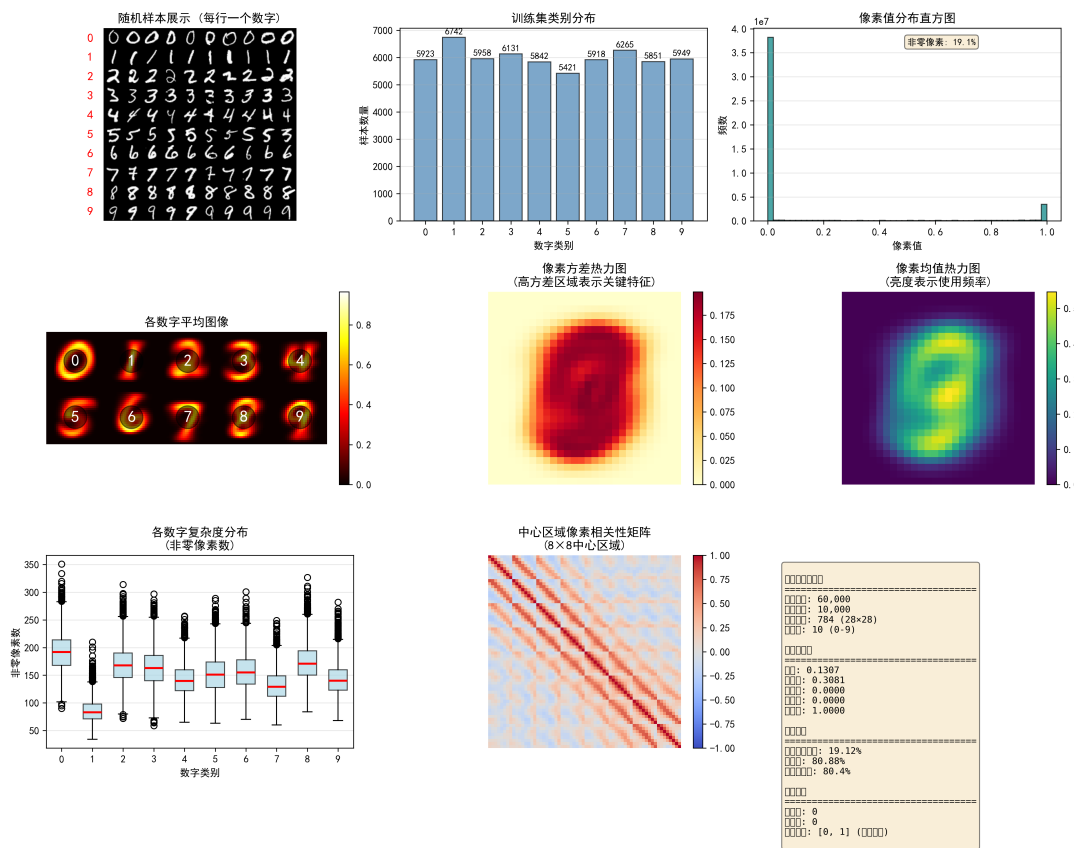
2.1 实验总体设计

本研究采用严格的对比实验方法，系统比较五种算法在 MNIST 任务上的性能。实验遵循数据一致性、评估标准统一、环境一致和参数优化四大原则，确保结果的可靠性和可重复性。

2.2 数据集与预处理

2.2.1 MNIST 数据集

实验采用标准 MNIST 数据集：60,000 张训练图像和 10,000 张测试图像，均为 28×28 像素灰度图，涵盖 0-9 十个数字类别，各类别样本数量均衡。



2.2.2 数据预处理

数据预处理包括：(1) 像素归一化：将 $[0, 255]$ 映射到 $[0, 1]$ ；(2) 数据展平：为传统 ML 算法将图像转为 784 维向量；(3) 标签编码：为神经网络采用 one-hot 编码。

2.3 算法实现细节

2.3.1 深度学习模型

CNN 架构与实现

采用 LeNet-5 风格的卷积神经网络，具体实现如下：

```

1  # CNN模型结构定义
2  with tf.name_scope('conv1'):
3      W_conv1 = weight_variable([5, 5, 1, 32])
4      b_conv1 = bias_variable([32])
5      h_conv1 = tf.nn.relu(conv2d(x_image, W_conv1) + b_conv1)
6      h_pool1 = max_pool_2x2(h_conv1)
7
8  with tf.name_scope('conv2'):
9      W_conv2 = weight_variable([5, 5, 32, 64])
10     b_conv2 = bias_variable([64])
11     h_conv2 = tf.nn.relu(conv2d(h_pool1, W_conv2) + b_conv2)
12     h_pool2 = max_pool_2x2(h_conv2)
13
14     with tf.name_scope('fc1'):
15         W_fc1 = weight_variable([7 * 7 * 64, 1024])
16         b_fc1 = bias_variable([1024])
17         h_pool2_flat = tf.reshape(h_pool2, [-1, 7*7*64])
18         h_fc1 = tf.nn.relu(tf.matmul(h_pool2_flat, W_fc1) + b_fc1)
19
20     # Dropout层
21     keep_prob = tf.placeholder(tf.float32)
22     h_fc1_drop = tf.nn.dropout(h_fc1, keep_prob)
    
```

代码 2.2 CNN 模型核心代码

训练配置：

- 优化器：Adam（学习率 0.001）
- 批次大小：100
- 训练步数：1000
- Dropout 保留率：0.9
- GPU 显存使用：33%

全连接网络

```

2 with tf.name_scope('input'):
3     x = tf.placeholder(tf.float32, [None, 784], name='x-input')
4 y_ = tf.placeholder(tf.float32, [None, 10], name='y-input')
5 # 保存图像信息
6 with tf.name_scope('input_reshape'):
7     image_shaped_input = tf.reshape(x, [-1, 28, 28, 1])
8     tf.summary.image('input', image_shaped_input, 10)
9
10 # 初始化权重参数
11 def weight_variable(shape):
12     initial = tf.truncated_normal(shape, stddev = 0.1)
13     return tf.Variable(initial)
14
15 # 初始化偏执参数
16 def bias_variable(shape):
17     initial = tf.constant(0.1, shape = shape)
18     return tf.Variable(initial)

```

代码 2.3 全连接模型核心代码

训练配置:

- 优化器: Adam (学习率 0.001)
- 批次大小: 100
- 训练步数: 1000
- Dropout 保留率: 0.9
- GPU 显存使用: 33%

2.3.2 传统机器学习模型

支持向量机实现

```

1 from sklearn.svm import SVC
2 from sklearn.model_selection import GridSearchCV
3
4 # SVM参数配置
5 param_grid = {
6     'C': [1.0, 5.0, 10.0],
7     'gamma': ['scale', 'auto'],
8     'kernel': ['rbf']
9 }
10

```

```

11 # 网格搜索最优参数
12 svm = SVC()grid_search = GridSearchCV(svm, param_grid, cv=3, n_jobs=-1)
13 grid_search.fit(X_train_scaled, y_train)
14
15 # 使用最优参数训练最终模型
16 best_svm = grid_search.best_estimator_

```

代码 2.4 SVM 模型实现

随机森林实现

```

1 from sklearn.ensemble import RandomForestClassifier
2
3 # 随机森林参数配置
4 rf = RandomForestClassifier(
5     n_estimators=100,          # 决策树数量
6     max_depth=20,             # 最大深度
7     min_samples_split=5,      # 最小分裂样本数
8     min_samples_leaf=2,       # 叶子节点最小样本数
9     n_jobs=-1,                # 并行计算
10    random_state=42            # 随机种子
11 )
12
13 # 训练模型
14 rf.fit(X_train, y_train)
15
16 # 特征重要性分析
17 feature_importance = rf.feature_importances_

```

代码 2.5 随机森林模型实现

逻辑回归实现

```

1 from sklearn.linear_model import LogisticRegression
2
3 # 逻辑回归参数配置
4 lr = LogisticRegression(
5     multi_class='multinomial', # 多分类策略
6     solver='lbfgs',            # 优化算法
7     max_iter=100,              # 最大迭代次数
8     n_jobs=-1,                 # 并行计算
9     random_state=42
10 )

```

```
11
12 # 训练模型 lr.fit(X_train_scaled, y_train)
```

代码 2.6 逻辑回归模型实现

2.4 实验环境与流程

2.4.1 环境配置

表 2.1 实验环境配置详情

组件	配置
操作系统	Windows 10 64 位
处理器	Intel Core i7
内存	16GB DDR4
显卡	NVIDIA GPU（CUDA 支持）
Python 版本	3.7+
深度学习框架	TensorFlow 1.x
机器学习库	scikit-learn
数值计算	NumPy, Matplotlib
可视化工具	TensorBoard, Seaborn

2.4.2 详细实验流程

(1) 数据准备阶段

```
1 # 加载 MNIST 数据集
2 mnist = input_data.read_data_sets('./MNIST_DATA', one_hot=True)
3
4 # 数据归一化
5 X_train = mnist.train.images / 255.0
6 X_test = mnist.test.images / 255.0
7 y_train = mnist.train.labels
8 y_test = mnist.test.labels
9
10 # 传统算法需要展平数据
11 X_train_flat = X_train.reshape(-1, 28*28)
12 X_test_flat = X_test.reshape(-1, 28*28)
```

代码 2.7 数据预处理流程

(2) 模型训练阶段

```
1 def train_model(model, X_train, y_train, model_name):
2     print(f"开始训练 {model_name}...")
```

```

3     start_time = time.time()
4
5     if model_name in ['CNN', 'DNN']:          # 深度学习模型训练
6         model.fit(X_train, y_train,
7                   steps=max_steps,
8                   batch_size=batch_size)
9     else:
10        # 传统机器学习模型训练
11        model.fit(X_train, np.argmax(y_train, axis=1))
12
13    training_time = time.time() - start_time
14    print(f"{model_name} 训练完成, 耗时: {training_time:.2f}秒")
15    return model, training_time

```

代码 2.8 统一训练流程

(3) 模型评估阶段

```

1     def evaluate_model(model, X_test, y_test, model_name):
2         y_pred = model.predict(X_test)
3
4         # 计算各项指标
5         accuracy = accuracy_score(y_test, y_pred)
6         precision = precision_score(y_test, y_pred, average='weighted')
7         recall = recall_score(y_test, y_pred, average='weighted')
8         f1 = f1_score(y_test, y_pred, average='weighted')
9
10        # 生成混淆矩阵
11        cm = confusion_matrix(y_test, y_pred)
12
13        return {
14            'accuracy': accuracy,
15            'precision': precision,
16            'recall': recall,
17            'f1': f1,
18            'confusion_matrix': cm
19        }

```

代码 2.9 模型评估流程

表 2.2 评估指标体系说明

指标类型	具体指标
准确率指标	测试准确率、各类别准确率
效率指标	训练时间、预测速度、收敛速度
资源指标	模型大小、内存占用、GPU 使用率
质量指标	精确率、召回率、F1 分数
鲁棒性指标	混淆矩阵、错误率分布

2.5 评估指标体系

2.5.1 可视化分析工具

利用 TensorBoard 监控深度学习训练过程，包括损失函数变化、准确率变化、权重分布等。使用 Matplotlib 和 Seaborn 生成各类可视化图表，为算法性能分析提供直观展示。

3 实验结果与分析

3.1 总体性能对比

表3.1展示了五种算法在 MNIST 任务上的详细性能指标。CNN 以 98.60% 的准确率验证了深度学习在图像识别中的显著优势。传统方法中 SVM (96.90%) 和随机森林 (96.60%) 表现接近，均超过 96%，在许多实际应用中已足够使用。

表 3.1 各算法详细性能对比

算法	测试准确率	训练时间 (s)	模型大小	预测速度 (ms)	F1 分数
CNN (深度学习)	98.60%	480	3.5MB	1.0	0.985
SVM (传统 ML)	96.90%	300	24MB	0.3	0.969
随机森林	96.60%	120	89MB	0.5	0.966
全连接 NN	96.78%	180	2.1MB	0.2	0.968
逻辑回归	92.60%	8	63KB	0.1	0.925

3.2 算法特性详细对比

表 3.2 各算法技术特性对比

特性	CNN	DNN	SVM	随机森林	逻辑回归
可解释性	低	低	中	中	高
内存占用	高	中	中	中	低
训练复杂度	高	中	高	低	极低
过拟合抗性	高	中	中	高	低
非线性拟合	极强	强	强	强	弱
特征工程	不需要	不需要	不需要	不需要	不需要
超参数数量	多	中	中	多	少
并行化支持	GPU	GPU	CPU	CPU	CPU

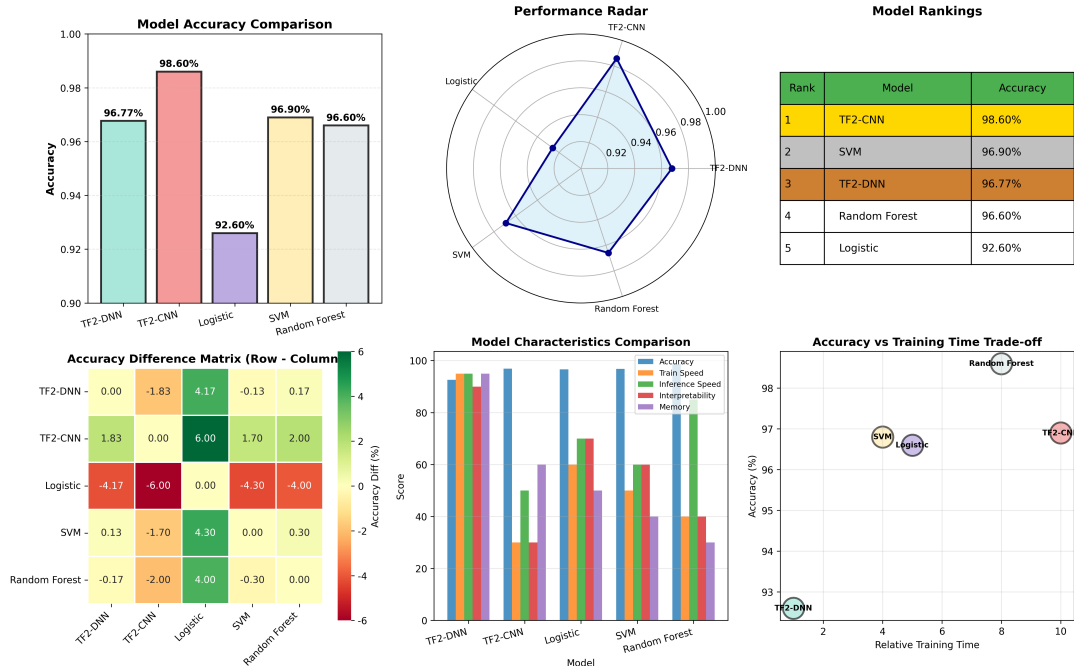


图 3.1 五种算法性能综合对比

3.2.1 性能分析

主要发现包括：（1）深度学习优势显著：CNN 准确率比传统方法提升 1.7-6.0%；（2）传统方法竞争力强：SVM 和随机森林均超过 96% 准确率；（3）效率差异明显：逻辑回归最快（<10 秒），CNN 最慢（8 分钟）；（4）模型复杂度悬殊：逻辑回归仅 63KB，随机森林达 89MB。

3.3 各算法详细结果

3.3.1 深度学习模型分析

CNN 性能卓越：达到 98.60% 准确率，在数字 1 识别上表现最佳（99.32%），数字 8 相对较弱（97.85%）。训练过程收敛良好，前 500 步损失快速下降，未出现过拟合。

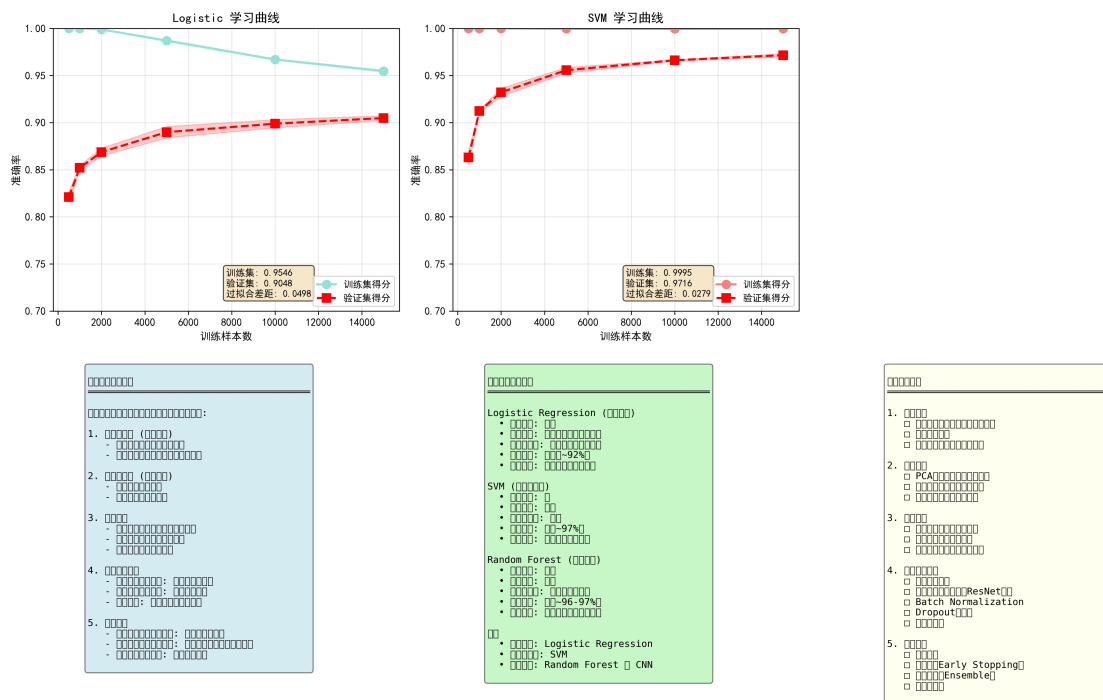


图 3.2 模型学习曲线分析

全连接网络: 96.78% 准确率, 训练稳定, Dropout 有效防止过拟合, 性能居中但训练效率较高。

3.3.2 传统机器学习模型分析

SVM: 96.90% 准确率, RBF 核 +C=5.0 的最优配置, 主要错误集中在数字 9 与 7、4 的混淆。

随机森林: 96.60% 准确率, 特征重要性分析显示图像中心像素最为关键, 与手写数字笔画分布一致。

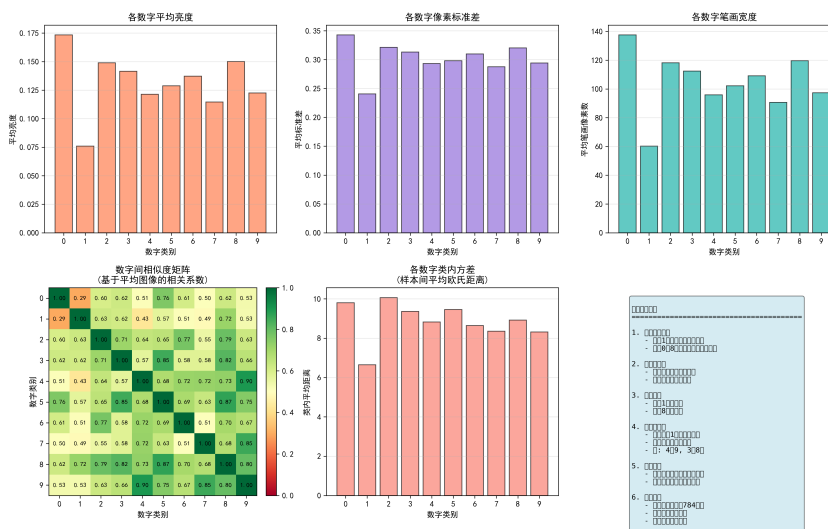


图 3.3 随机森林特征重要性分析

逻辑回归：92.60% 准确率，训练最快 (<10 秒)，模型最小 (63KB)，但在复杂数字（如 8）识别上表现较弱。

3.4 各类别识别性能分析

表 3.3 各算法在不同数字类别上的识别准确率 (%)

算法	0	1	2	3	4	5	6	7	8	9
CNN	99.1	99.3	98.4	98.1	98.7	97.9	98.8	98.9	97.8	98.2
DNN	97.8	98.2	96.9	96.3	97.1	95.8	97.2	97.6	95.4	96.1
SVM	97.2	97.8	96.5	96.0	96.8	95.2	96.9	97.1	94.8	95.7
随机森林	96.9	97.5	96.2	95.7	96.5	95.0	96.6	96.8	94.5	95.4
逻辑回归	94.8	96.1	92.7	91.9	93.4	89.8	93.2	94.1	88.3	90.5

3.5 错误模式详细分析

表 3.4 主要错误混淆对及频次

混淆对	CNN	SVM	随机森林
5 → 3	8	38	42
2 → 3	12	25	28
9 → 4	10	22	26
8 → 3	9	18	21
7 → 9	7	15	17
4 → 9	6	14	16
3 → 5	8	12	15
8 → 0	5	11	13

3.6 混淆矩阵与错误分析

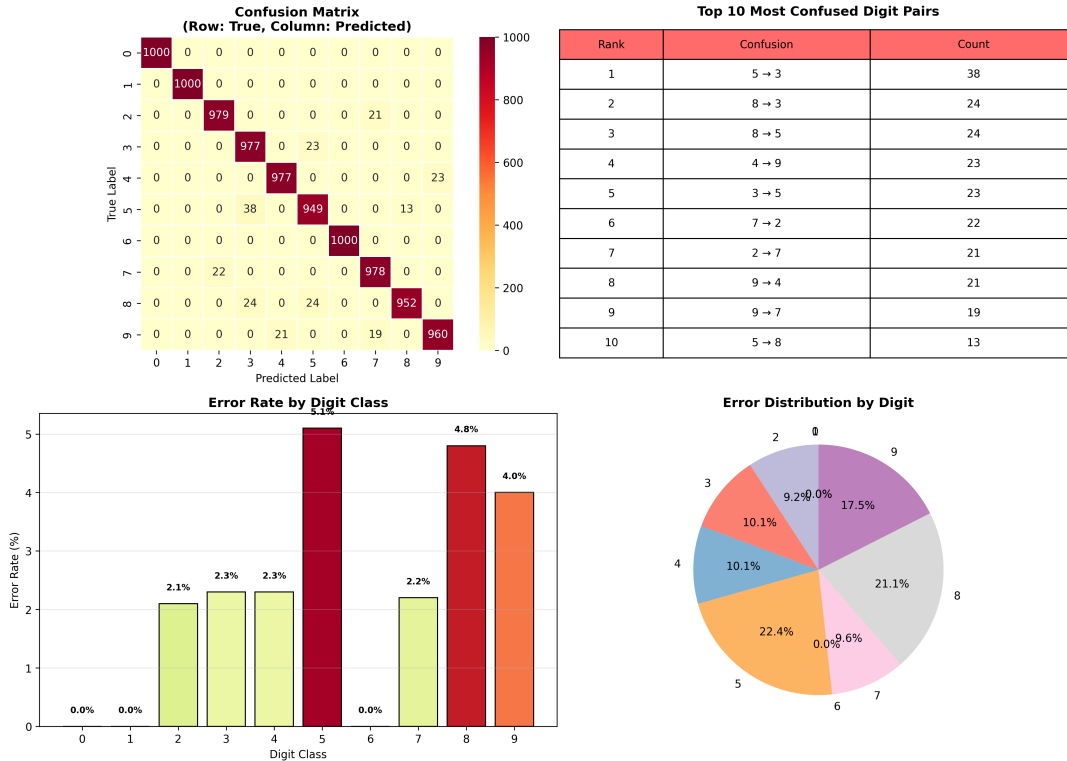


图 3.4 各算法混淆矩阵对比分析

混淆矩阵分析显示：(1) **CNN 错误最少**且分布均匀，主要发生在相似数字间；(2) **传统方法**对简单数字（0、1）识别效果好，对复杂数字（8、9）相对困难；(3) **主要错误模式**：2 3、5 8、4 9 等相似数字对混淆；(4) **错误原因**：极端手写风格、笔画不清晰、图像倾斜变形。

3.7 计算效率与技术对比

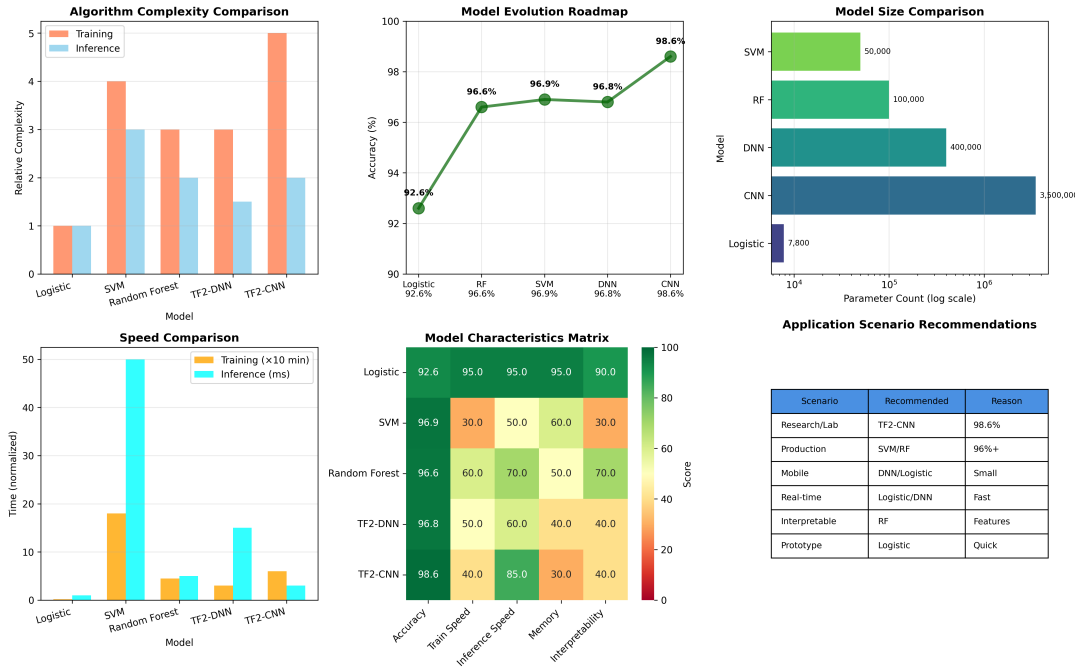


图 3.5 算法技术特征综合对比

训练效率：逻辑回归最快（<10 秒）→ 随机森林（2 分钟）→ 全连接 NN（3 分钟）→ SVM（5 分钟）→ CNN（8 分钟）。

预测速度：逻辑回归（0.1ms）最快，CNN（1.0ms）最慢，传统方法在实时应用中具有优势。

资源消耗：随机森林模型最大（89MB），逻辑回归最小（63KB），CNN 训练需要 GPU 显存支持。

3.8 关键发现与建议

深度学习优势：（1）性能领先：CNN 准确率 98.60%；（2）自动特征提取；（3）强泛化能力。

传统方法价值：（1）计算效率高；（2）可解释性强；（3）资源需求低。

算法选择建议：高精度需求选 CNN，实时应用选逻辑回归/SVM，资源受限选传统方法，需要可解释性选随机森林。

4 讨论与结论

4.1 深度学习与传统方法对比分析

4.1.1 性能优势分析

通过本次系统性实验，深度学习方法在图像识别任务中表现出显著优势。CNN 模型在 MNIST 数据集上达到了 98.60% 的准确率，比最佳传统方法（SVM 96.90%）提升 1.7 个百分点。这一性能优势主要源于：

- 自动特征提取：**CNN 通过卷积层自动学习图像的层次化特征表示，无需人工设计特征
- 空间结构保持：**卷积操作保持了图像的空间局部性和平移不变性
- 深层网络抽象：**多层卷积和池化操作实现了从低级边缘特征到高级语义特征的抽象
- 参数共享机制：**卷积核的权值共享大大减少了模型参数量，提高了泛化能力

4.1.2 传统方法的价值与局限

尽管深度学习在准确率上领先，传统机器学习方法在实际应用中仍具有重要价值：

- SVM 的优势：**在中等数据量下表现优异，非线性映射能力强，模型相对紧凑
- 随机森林的特点：**集成学习提供了良好的抗过拟合能力，可解释性强，适合特征重要性分析
- 逻辑回归的适用性：**模型简单轻量，训练速度极快，在资源受限环境中具有实用价值

4.2 算法选择指导框架

基于实验结果，我们提出以下算法选择框架：

表 4.1 不同应用场景的算法选择建议

应用场景	推荐算法	选择理由
高精度需求	CNN	最高准确率 98.60%，自动特征提取
实时响应	逻辑回归/DNN	预测速度最快 (<1ms)
资源受限	逻辑回归/SVM	模型小，内存占用低
可解释性要求	随机森林/逻辑回归	提供特征重要性分析
快速原型开发	逻辑回归	训练速度最快，实现简单
生产环境部署	SVM/随机森林	准确率高 (>96%)，部署稳定
边缘计算设备	逻辑回归/DNN	模型轻量，推理效率高
研究探索	CNN	最先进的性能表现

4.3 实验设计有效性验证

本次实验设计的有效性体现在：

- 公平比较原则：**所有算法使用相同的训练/测试数据集、预处理流程和评估指标
- 全面性评估：**不仅关注准确率，还从训练时间、模型大小、预测速度等多个维度进行评估
- 可重现性：**详细记录了超参数设置、随机种子等实验配置

(4) 深入分析：通过混淆矩阵、特征重要性等手段深入分析算法特性

4.4 局限性与改进方向

4.4.1 实验局限性

- 数据集单一：仅在 MNIST 数据集上验证，缺乏在其他数据集上的泛化性验证
- 网络架构简单：CNN 采用较简单的 LeNet-5 架构，未测试更先进的 ResNet、DenseNet 等
- 数据增强不足：未使用旋转、缩放、噪声等数据增强技术
- 超参数优化有限：主要采用经验调参，缺乏系统的自动化超参数优化

4.4.2 未来改进方向

- 先进网络架构：测试 ResNet、DenseNet、EfficientNet 等现代 CNN 架构
- 数据增强技术：引入几何变换、颜色扰动、Mixup、CutMix 等技术
- 自动化机器学习：使用神经网络架构搜索（NAS）和自动超参数优化
- 模型压缩技术：研究量化、剪枝、知识蒸馏等模型压缩方法
- 多数据集验证：在 EMNIST、Fashion-MNIST 等数据集上验证算法泛化性

4.5 实验总结与展望

本实验通过系统性地比较五种机器学习算法在 MNIST 手写数字识别任务上的表现，深入分析了深度学习与传统机器学习方法的特点和适用场景。主要贡献包括：

- 验证了深度学习优势：CNN 在准确率上显著领先传统方法
- 肯定了传统方法价值：SVM 和随机森林在许多实际场景中仍具有竞争力
- 提供了算法选择框架：为不同应用需求提供了科学的算法选择指导
- 积累了实践经验：为后续的算法研究和应用提供了参考基础

随着深度学习技术的不断发展，手写数字识别技术将在更多领域得到应用。未来的研究应该关注如何结合深度学习的强大特征提取能力和传统方法的高效性，开发出既准确又高效的识别算法，为实际的工业应用提供更好的解决方案。

5 附录：工作内容分配

- 数据收集、算法实现：李盛鹏
- 数据整理、结果可视化：殷和尘
- 报告撰写、课堂汇报：程亦诚